

- [TradePilot – Strategy Research: Profitability, Usability, Platform](#)
 - [0. The one finding that ties all three pillars together](#)
 - [1. Pillar A – Make the engines more profitable](#)
 - [2. Pillar B – Make it user-friendly](#)
 - [3. Pillar C – Make it an agentic platform](#)
 - [4. The synthesis – what to actually pursue](#)
 - [5. Open research questions \(next, if you want to go deeper\)](#)

TradePilot – Strategy Research: Profitability, Usability, Platform

Research-only. Three pillars – make the engines more profitable, make it user-friendly, make it an agentic platform – and the single thread that ties them together.

Project	TradePilot (autonomous agentic trading)
Document	Strategy research – 3-pillar synthesis (no implementation)
Version	v1.0.0
Status	Research complete
Created	2026-06-30
Method	3 parallel research agents (UX · profitability · platform), each grounded in our prior validated findings + external literature
Author	Soumya Swain
Email	soumya@sidewall.in

0. The one finding that ties all three pillars together

The three research threads were run independently. They converged on **the same two truths**:

1. **Simplicity beats complexity – everywhere.** Profitability: a 5-tree model out-hits the 1,735-tree model; execute-at-open beats the rescore loop. UX: a single status surface beats the 10-tab dashboard. Platform: white-box beats black-box. Every pillar's top lever is *remove complexity*, not add it.
2. **The explainable decision pipeline is one artifact with four payoffs.** The “why did it trade X” decision log is simultaneously: the **profitability** validation (auditable, white-box), the **UX** trust surface, the **compliance** answer (white-box avoids the SEBI RA licence + satisfies the audit trail), and the **platform** architecture (the audit/memory agents). Build it once, win four times.

And a structural bonus: the frontier agent architecture (Pillar 3) **prevents the overfitting that's hurting profitability** (Pillar 1) – alpha agents never see evaluation-window returns; memory separates train/eval namespaces. The platform upgrade and the profitability fix are the *same* investment.

1. Pillar A – Make the engines more profitable

Every internal finding was independently confirmed by external quant literature. The edge is real (v5 alpha t=3.0, cost-robust to ~45bps) but destroyed at the execution and model-complexity layer. Five levers, ranked:

#	LEVER	LEVERAGE / EFFORT
1	Execute at open, kill the rescore loop – lock prior-close scores, enter 09:15-09:20, hold	Highest · zero cost (code)
2	Collapse the ML model to ≤20 trees + purged walk-forward (embargo folds, 6-8 OOS windows)	High · 2-3 wks
3	ATR-proportional sizing, ≥60% deployment (fractional Kelly on the validated edge)	High · changes return profile now
4	Binary regime gate on <i>direction</i> (NIFTY 20-day SMA + VIX-rank → disable shorts on weak days)	Medium · drawdown control
5	Retrain ML on the correct label – P(hit +1.5% before -0.75%), not open→close return	Medium · fixes label/exit mismatch

External backing (selected):

- **Overfitting is existential.** Bailey & Lopez de Prado: after ~4.75 independent backtests, a Sharpe of 1.0 is achievable *by pure chance*. A 1,735-tree model tuned over hundreds of experiments has a meaningless undeflated Sharpe. ScienceDirect (2024): complex models hit ~50% out-of-sample accuracy with “chaotic” hyperparameter sensitivity – the fingerprint of overfit. *Simpler models generalize because they can’t memorize enough to fit noise.*
- **Alpha decays with execution lag.** Man Group: a signal’s value falls as the lag between generation and execution grows. A morning-momentum signal has a half-life in *minutes*; entering 30-60 min late (the rescore loop) chases exhausted moves.
- **Underbetting destroys returns as surely as overbetting** (Kelly). 35% deployment on a t=3.0 edge leaves half the return on the table; half-Kelly keeps ~75% of growth at half the variance.
- **Regime filters give modest gains, not magic** – and only if kept simple. A filter that fires 15 times in 2 years can’t be optimized without curve-fitting. Gate *direction of the book*, not individual entries; two unoptimised signals (SMA + VIX-rank).

What to AVOID (literature-backed): more model features (noise amplifiers), optimising regime params, re-expanding to NIFTY-200 without a separate mid-cap edge, leverage to mask under-deployment, a second intraday entry window (churn disguised as diversification), in-sample validation, standard K-fold (leaks future data – use purged folds).

2. Pillar B – Make it user-friendly

The reframe: **in an agentic product the user is a supervisor, not a pilot.** The UX job is to make supervision safe, legible, and low-friction – the opposite of a feature-dense builder.

MOVE	WHAT IT MEANS	WHY
Lead	Single status surface – “is the agent running? what did it do today? anything unusual?”	Replaces 10 tabs; Linear’s “is everything okay?” pattern
Lead	Plain-language decision log per trade (“bought RELIANCE – momentum + volume, 0.5% size, stop 2455”)	The product, not cosmetics; also the SEBI audit trail
Lead	Progressive-delegation onboarding – Observe → Approve-first → Supervised-auto → Full-auto	Users who <i>earn</i> full delegation keep it; day-one autonomy gets abandoned
Add	“Why did it do that?” per-trade explainer; drawdown/risk display (not just P&L)	Users tolerate losses they expected, not ones they didn’t know were possible
Add	Weekly performance narrative via Telegram; progress-vs-Nifty framing	Less panic-selling than raw daily P&L
Cut	The 10-tab nav, the A/B engine toggle, real-time ticker feeds – into a builder-only mode	A user watching tickers isn’t trusting the agent

Telegram = **check-not-operate**: end-of-session summary + anomaly alert + intervention-request only. “Timing beats intelligence” – never trade-by-trade noise. Keep the current builder view via a **dual-mode UI** (builder vs user), not a rewrite.

3. Pillar C – Make it an agentic platform

Recommended product shape for India: “BYOB White-Box Agent” – *your own autonomous AI agent running in your Zerodha/Fyers account; set your mandate once; it researches, decides, and trades; you see every reasoning step.* The platform holds **zero capital** (no PMS/fiduciary licence); **white-box** sidesteps the heavy black-box RA gate. eToro’s Agent Portfolios proved the shape; [india-trade-cli](#) (7-agent OSS, Zerodha+Fyers live) proves it’s buildable indie-scale.

T3 → T4 architecture upgrade – decompose the monolith into a 9-role agent DAG:

LAYER	AGENTS	KEY PROPERTY
Strategy	Planner + Meta-agent	Rewrites agent prompts/code from outcomes (the T4 unlock)
Signal	Alpha agents	Never see evaluation-window returns (anti-overfit)
Control	Risk-gate agents	Binary block , never modify signals
Build	Portfolio, Backtest, Execution	Deterministic solvers; cost/slippage accounting
Trust	Audit + Memory agents	UUID-indexed, immutable, train/eval namespaces

Multi-tenant requirements: per-user state/ledger/IPS/memory namespace; per-user broker creds (scoped API keys); per-user mandate (Investment Policy Statement) the orchestrator reads before dispatch; audit trail as a product primitive (SEBI: Client → Algo-ID → Static IP → API key, 5-yr retention).

Phased path:

PHASE	WHEN	WHAT	COMPLIANCE
0 – Personal T4	now → 6mo	9-role DAG, meta-agent, UUID memory, explainability log	Zero (paper/own capital)
1 – Invite-only BYOB	6-12mo	3-10 trusted users, per-user IPS + broker vault + static IP, white-box algo-ID	Light (white-box, no RA licence)
2 – Closed beta	12-18mo	50-200 users, subscription, copy-agent of our own variants	SEBI RA licence, ISO 27001 prep
3 – Platform	18-24mo	third-party agent SDK + marketplace	Empanelment, VAPT, ISO 27001

4. The synthesis – what to actually pursue

★ **Insight** — The three pillars are not three projects – they share a spine. **Profitability says “simplify the model and execute clean”; the platform architecture says “decompose into agents whose alpha layer can’t see the future”; the UX says “show the user every decision.”** All three are satisfied by one build: a **simple, explainable, clean-execution decision pipeline**. Shrink the model (profit), wrap each decision in a logged rationale (UX trust + compliance), and structure it as auditable agents with train/eval separation (platform + anti-overfit). You don’t choose between the three pillars – you build the spine and all three light up.

The sequence the research implies (still research – not a build order to execute yet): 1. **Profit first, cheaply:** Levers 1-3 (execute-at-open, shrink the model, deploy capital) are mostly code + retrain, zero compliance, and prove the engine before anything is productised. 2. **Then the explainable pipeline:** build the decision log / white-box rationale – it’s the UX trust surface *and* the compliance artifact *and* the platform’s audit layer, all at once. 3. **Then phased productisation:** Phase-0 personal T4 architecture → invite-only BYOB → product, with compliance scaling per user exactly as our SEBI research predicted.

5. Open research questions (next, if you want to go deeper)

#	QUESTION	PILLAR
Q1	Capacity curve for v5 – how does slippage scale with lot size (order/30-day-ADV)? Sets the ceiling on capital deployment	Profit
Q2	Does <code>backtest-honest-fills.py</code> use purged folds or leaky K-fold? Quantify OOS-Sharpe inflation before trusting the 5-tree “win”	Profit
Q3	Short-book regime regression – on losing-short days, what was NIFTY return + VIX vs median? Confirms the binary regime gate	Profit
Q4	Indian retail-trader interviews (5-10) – what does a non-builder fear about an agent trading their capital?	UX
Q5	Telegram approval-flow latency (P50/P95) during market hours – does approve-first need a fallback default?	UX
Q6	Reconcile the two existing long-only experiments (<code>v5_long</code> vs <code>tradepilot-v5-longonly-ab</code>)	Platform/Profit